# Laboratory assignment

**Component** 5 and 6

**Authors:** Patricia Moga, Ciobanu Sergiu-Tudor
**Group:** 5

May 28, 2026

# 1 Conceptual Modeling: The PAGES System

## 1.1 PAGES for each Agent

Each agent within the system acts as a distinct entity with specific roles and capabilities:

Table 1: PAGES model per agent type

| Component | Description |
|---|---|
| **P – Perception** | *Strategic Player:*<br>• Full 9×9 board state.<br>• Status of each local board (won / lost / draw).<br>• Active local board constraint (from last move).<br>• Turn-request signal written to the blackboard from Game Master.<br><br>*Game Master:*<br>• Move coordinates $(x, y)$ written to the Blackboard.<br>• Current board state and constraints. |
| **A – Actions** | *Strategic Player:*<br>• Select a cell $(x, y)$ within the active local board.<br>• Output a heuristic score.<br><br>*Game Master:*<br>• Validate submitted move against rules.<br>• Update global board state.<br>• Declare win / draw / continue game. |

**Table 1 – Continued from previous page**

| Component | Description |
|---|---|
| **G – Goals** | *Strategic Player:*<br>• Win three local boards in a row on the global grid.<br>• Direct opponent to unfavourable local boards.<br>• Maximise $V(s)$ with Minimax + Alpha-Beta Algorithm.<br><br>*Game Master:*<br>• Keep the gameplay fair and strictly within the rules.<br>• Ensure fair turn-taking between agents. |
| **E – Environment** | • the 9×9 hierarchical Ultimate Tic-Tac-Toe grid. |
| **S – State of Environment** | • Idle — waiting for turn signal.<br>• Active — reading board state / running Minimax / writing move.<br>• checking move legality<br>• game-over condition reached |

### 1.2 PEAS Description

| Component | Description |
|---|---|
| P – Performance Measure | - Winning the game or achieving a draw<br>– Heuristic board evaluation score<br>– Local boards captured vs opponent |
| E – Environment | – 9×9 hierarchical game grid, the Xs and Os on the local board placed by the oponent, and the available empty spaces<br>– Adversarial between players, rule-governed by Game Master |
| A – Actuators | – Blackboard write interface (move coordinates / board state): placing Xs and Os in one of the empty space on the board |
| S – Sensors | – Blackboard reader (full board state + local board statuses)<br>– Rule checker / win-condition detector |

Table 2: PEAS description

| Property | Type | Motive |
| --- | --- | --- |
| Accessible vs Inaccessible | **Accessible** | Full read access to the complete 9×9 board state, local board statuses, and active board constraint. |
| Deterministic vs Non-deterministic | **Deterministic** | Every move the player makes has a predictable outcome. The same move always leads to the same result. There is no randomness or uncertainty in how the board changes after an action. |
| Episodic vs Non-episodic | **Non-episodic** | Each move affects the next one. Where a player places their mark decides which local board the opponent must play in, so decisions are connected across the whole game. |
| Static vs Dynamic | **Static** | The board does not change while an agent is calculating its best move. It only updates after a move is submitted and approved by the Game Master. |
| Discrete vs Continuous | **Discrete** | The state space and action space are finite. There are exactly 81 cells, each with a finite set of values (empty, X, O), and actions are integer coordinate pairs (x, y). |
| Markovian vs Non-Markovian | **Markovian** | The current state is not influenced by past moves. The current board state combined with the active local board constraint fully encodes all information needed to determine future moves. |

Table 3: Environment properties of the Ultimate Tic-Tac-Toe MAS

### 1.3 Properties of the Environment

## 2 Design of the MAS

### 2.1 Agents' Role within the system

- **Strategic Agents (Player 1 and Player 2)**
  - **Role:** Autonomous, concurrent decision-making agents.
  - **Goal:** Win three local boards in a row on the global grid.
  - **Perception:** Full board state and active local board constraint read from the Blackboard.
  - **Actions:** Select and submit the optimal cell (x, y) via Minimax + Alpha-Beta pruning.
  - **Collaboration:** Reads board state written by the Game Master from the Blackboard.
- **Game Master Agent**
  - **Role:** Central coordinator responsible for game integrity.
  - Does not perform any strategic reasoning or planning.

- **Utility:**
  * Writes the current board state and active turn flag to the Blackboard.
  * Reads and validates moves submitted by Strategic Agents.
  * Updates the board and determines win / draw / continue conditions.

### 2.2 Agents' Architecture

| **Property** | **Type** | **Motive** |
|---|---|---|
| Strategic Agent (P1 / P2) | **Deliberative** | Reads the current board state and plans ahead by exploring possible future moves via Minimax search before committing to a decision. |
| Game Master | **Reactive** | Waits for a move to appear on the Blackboard, then validates it, updates the board, applies the redirection rule, and flips the turn flag. |

Table 4: Agent architectures

### 2.3 Communication and Interaction Model

- **Communication Type:** Indirect via a Blackboard system.
- **Architecture:** Centralized communication, distributed control.
- **Access Model:**
  - Game Master: read and write
  - Strategic Agents: read and write
- **Workflow:**
  1. Game Master writes the current board state and sets `current_turn` to the active player.
  2. Active Strategic Agent reads the Blackboard, detects its turn via the flag, and runs Minimax.
  3. The Strategic Agent writes its chosen move coordinates (x, y) to the Blackboard.
  4. Game Master reads the move, validates it, updates the board state, and flips `current_turn` to the other player.
  5. Steps 1–4 repeat until a win or draw condition is detected.
- **Coordination:**
  - Between players: competitive (each tries to maximize its own heuristic and finally win the game).
  - With Game Master: cooperative (players rely on it for state updates and turn management).